



Interface Contract v1

👤 Created by	ⓑ BTG 47
🕒 Created time	April 9, 2026 7:24 PM
☰ Category	Especificación técnica
👤 Last edited by	ⓑ BTG 47
🕒 Last updated time	April 9, 2026 9:34 PM
⚙️ Status	Hecho

Interface Contract v1

CNN + DeepProbLog Integration Mock

Focus Case: Mosquito

1. Purpose

This document defines the **first interface contract** between the visual model and the reasoning module for the insect classification project.

The purpose of this contract is to let all team members work in parallel while sharing a common expectation of:

- what the CNN should receive,
- what the CNN should output,
- what DeepProbLog should consume,
- what the reasoning layer should return,
- and how the mosquito example should look end-to-end.

This version is a **mock contract**, not the final production schema.

It is intended to provide a stable starting point for development and discussion.

2. Architectural Intent

The current architecture is divided into four conceptual stages:

1. **Image Input**
2. **CNN Visual Evidence Extraction**
3. **DeepProbLog Reasoning**
4. **Optional DPLA* Approximate Inference Layer**

For the current MVP, the strongest focus is the connection:

image → CNN evidence → DeepProbLog query/result

DPLA* is acknowledged in the architecture but is not the main deliverable of this first integration phase. DeepProbLog is the component that combines neural outputs with probabilistic logic through neural predicates, while DPLA* is useful later when exact reasoning becomes too expensive.

3. Scope of v1

This contract is intentionally limited.

Included

- one image at a time
- one visual prediction package per image
- top-k class predictions
- trait probabilities
- visibility flags
- DeepProbLog-compatible evidence
- final reasoning output
- mosquito as the main worked example

Not included yet

- real-time video
 - segmentation masks
 - prototype-based explainability
 - collection-scale taxonomy
 - exact DPLA* behavior in production
 - advanced uncertainty calibration
-

4. Initial Class Space

For the current MVP, the system assumes a reduced class space of **10 visually distinct insect classes**.

Suggested labels:

- dragonfly
- grasshopper
- praying_mantis
- cockroach
- stink_bug
- lady_beetle
- green_lacewing
- honey_bee
- monarch_butterfly
- mosquito

This reduced setup is intentional so the team can first validate the bridge between CNN output and logical inference before scaling.

5. Morphological Vocabulary v1

The CNN is not expected to produce only a final class.

It should also produce probabilities for a **closed vocabulary of traits**.

This is biologically reasonable because insect identification often depends on visible morphology such as wings, mouthparts, antennae, body form, and leg specialization.

Trait groups

```
{
  "wing_count": ["one_pair", "two_pairs", "reduced", "unknown"],
  "forewing_type": ["elytra", "hemelytra", "membranous", "scaly", "tegmina", "unknown"],
  "mouthpart_type": ["chewing", "piercing_sucking", "siphoning", "lapping_sponging", "unknown"],
  "antenna_type": ["filiform", "clavate", "plumose", "setaceous", "moniliform", "unknown"],
  "leg_specialization": ["jumping", "grasping", "running", "generalist", "unknown"],
  "body_shape": ["elongated", "rounded_domed", "shield_like", "slender", "oval_flat", "unknown"],
  "waist_shape": ["narrow_waist", "no_waist", "unknown"]
}
```

6. Global Pipeline Contract

Stage A — Input to CNN

The visual module receives one image and optional lightweight metadata.

Stage B — Output from CNN

The visual module returns:

- top-1 prediction
- top-k predictions
- trait probability heads
- visibility flags
- confidence metadata

Stage C — Input to DeepProbLog

The reasoning module receives a transformed version of the CNN output:

- candidate classes or orders
- trait probabilities
- visibility information
- quality information

Stage D — Output from DeepProbLog

The reasoning module returns:

- final class hypothesis
- reasoning trace
- activated rules
- confidence/score
- notes about uncertainty

Stage E — Optional DPLA* output

Later, the inference layer may also expose:

- best proof
- alternative proofs
- approximate search statistics

DPLA* is best understood here as an inference support mechanism, not as an independent classifier.

7. Input Schema v1

7.1 Raw Image Record

```
{  
  "image_id": "img_000887",  
  "file_name": "img_000887.jpg",  
  "file_path": "dataset/test/mosquito/img_000887.jpg",  
}
```

```

"split": "test",
"source": "local_dataset_v1",
"label_target": {
  "common_name": "mosquito",
  "order": "Diptera",
  "family": "Culicidae"
},
"metadata": {
  "view": "lateral",
  "background_type": "plain",
  "quality": "good",
  "bbox_available": false
}
}

```

Field notes

- `image_id` must be unique.
- `label_target` is available only for supervised data.
- `metadata.view` is useful because some traits may be hidden depending on the angle.

8. CNN Output Schema v1

This is the **main output contract** for the vision module.

```

{
  "image_id": "img_000887",
  "cnn_output": {
    "top_1": {
      "label": "mosquito",
      "order": "Diptera",
      "family": "Culicidae",
      "probability": 0.74
    },
    "top_k": [
      {
        "label": "mosquito",
        "order": "Diptera",
        "family": "Culicidae",
        "probability": 0.74
      },
      {
        "label": "green_lacewing",

```

```

        "order": "Neuroptera",
        "family": "Chrysopidae",
        "probability": 0.12
    },
    {
        "label": "dragonfly",
        "order": "Odonata",
        "family": "Libellulidae",
        "probability": 0.08
    }
]
},
"trait_heads": {
    "wing_count": {
        "one_pair": 0.89,
        "two_pairs": 0.08,
        "reduced": 0.00,
        "unknown": 0.03
    },
    "forewing_type": {
        "elytra": 0.00,
        "hemelytra": 0.01,
        "membranous": 0.91,
        "scaly": 0.01,
        "tegmina": 0.00,
        "unknown": 0.07
    },
    "mouthpart_type": {
        "chewing": 0.03,
        "piercing_sucking": 0.78,
        "siphoning": 0.01,
        "lapping_sponging": 0.06,
        "unknown": 0.12
    },
    "antenna_type": {
        "filiform": 0.22,
        "clavate": 0.01,
        "plumose": 0.58,
        "setaceous": 0.04,
        "moniliform": 0.00,
        "unknown": 0.15
    },
    "leg_specialization": {
        "jumping": 0.01,
        "grasping": 0.00,
        "running": 0.10,
        "generalist": 0.71,
        "unknown": 0.18
    },
    "body_shape": {
        "elongated": 0.11,
        "rounded_domed": 0.00,

```

```
    "shield_like":0.00,  
    "slender":0.87,  
    "oval_flat":0.00,  
    "unknown":0.02  
  },  
  "waist_shape": {  
    "narrow_waist":0.04,  
    "no_waist":0.70,  
    "unknown":0.26  
  }  
},  
"visibility_flags": {  
  "head_visible":true,  
  "antennae_visible":true,  
  "wings_visible":true,  
  "legs_visible":true,  
  "body_complete":true  
},  
"quality": {  
  "usable_for_reasoning":true,  
  "quality_score":0.91  
}  
}
```

9. CNN Output Design Rules

Rule 1

The CNN must always return **top-1** and **top-k** predictions.

Rule 2

The CNN must always return **all trait groups**, even if many values are low.

Rule 3

Every trait group must sum approximately to 1.0.

Rule 4

If a structure is not visible, the model should reflect that through:

- `visibility_flags`
- and/or a higher `unknown` value in the relevant trait head

Rule 5

The reasoning layer should not depend on one single number only.

It should consume a **bundle of evidence**.

This reflects the intended use of DeepProbLog: combine neural evidence with logical structure, rather than just post-process a hard label.

10. DeepProbLog Input Schema v1

The reasoning module does not need raw images.

It consumes structured evidence.

```
{
  "image_id": "img_000887",
  "evidence_for_reasoner": {
    "candidate_classes": [
      ["mosquito", 0.74],
      ["green_lacewing", 0.12],
      ["dragonfly", 0.08]
    ],
    "candidate_orders": [
      ["Diptera", 0.74],
      ["Neuroptera", 0.12],
      ["Odonata", 0.08]
    ],
    "traits": [
      ["one_pair", 0.89],
      ["membranous", 0.91],
      ["piercing_sucking", 0.78],
      ["plumose", 0.58],
      ["slender", 0.87],
      ["no_waist", 0.70]
    ],
    "visibility": {
      "head_visible": true,
      "antennae_visible": true,
      "wings_visible": true,
      "legs_visible": true,
      "body_complete": true
    },
    "quality": {
      "usable_for_reasoning": true,
      "quality_score": 0.91
    }
  }
}
```

```
}  
}
```

11. DeepProbLog Logic Representation Mock

A practical intermediate representation can also be expressed as facts.

```
cnn_class(img_000887, mosquito, 0.74).  
cnn_class(img_000887, green_lacewing, 0.12).  
cnn_class(img_000887, dragonfly, 0.08).  
  
cnn_order(img_000887, diptera, 0.74).  
cnn_order(img_000887, neuroptera, 0.12).  
cnn_order(img_000887, odonata, 0.08).  
  
trait(img_000887, one_pair, 0.89).  
trait(img_000887, membranous, 0.91).  
trait(img_000887, piercing_sucking, 0.78).  
trait(img_000887, plumose, 0.58).  
trait(img_000887, slender, 0.87).  
trait(img_000887, no_waist, 0.70).  
  
visible(img_000887, head).  
visible(img_000887, antennae).  
visible(img_000887, wings).  
visible(img_000887, legs).  
visible(img_000887, body_complete).  
  
quality_score(img_000887, 0.91).  
usable_for_reasoning(img_000887).
```

12. Mosquito Rules Mock v1

These are not final biological rules.

They are **engineering rules for MVP integration**.

```
candidate_order(I, diptera) :-  
    trait(I, one_pair, P1), P1 > 0.75.  
  
candidate_order(I, diptera) :-  
    cnn_order(I, diptera, P1), P1 > 0.60.
```

```

candidate_family(I, culicidae) :-
    candidate_order(I, diptera),
    trait(I, piercing_sucking, P2), P2 > 0.60,
    trait(I, slender, P3), P3 > 0.70.

final_class(I, mosquito) :-
    candidate_family(I, culicidae),
    cnn_class(I, mosquito, P4), P4 > 0.50.

final_class(I, mosquito) :-
    candidate_order(I, diptera),
    trait(I, plumose, P5), P5 > 0.40,
    trait(I, piercing_sucking, P6), P6 > 0.60.

```

Why these make sense for v1

The goal here is not to encode all entomology perfectly.

The goal is to demonstrate:

- the CNN proposes evidence,
- DeepProbLog consumes it,
- the final decision is supported by a traceable rule path.

13. DeepProbLog Output Schema v1

```

{
  "image_id": "img_000887",
  "reasoning_output": {
    "final_prediction": {
      "label": "mosquito",
      "order": "Diptera",
      "family": "Culicidae",
      "score": 0.84
    },
    "activated_rules": [
      "candidate_order_from_one_pair",
      "candidate_family_culicidae_from_piercing_sucking_and_slender",
      "final_class_mosquito_from_family_and_cnn_support"
    ],
    "reasoning_trace": [
      "one_pair (0.89) strongly supports Diptera",
      "piercing_sucking (0.78) is compatible with a mosquito-like candidate",
      "slender body (0.87) supports the Culicidae hypothesis",
      "CNN top-1 mosquito (0.74) reinforces the final class"
    ]
  }
}

```

```

    ],
    "reasoning_summary": "The system predicts mosquito because the image evidence strongly
    supports a Diptera-like specimen with one visible wing pair, piercing-sucking mouthparts,
    a slender body, and CNN support for mosquito.",
    "uncertainty_notes": [
    "plumose antenna confidence is moderate, not dominant",
    "family assignment is partly supported by class-level CNN evidence"
    ]
  }
}

```

14. Optional DPLA* Output Schema v1

This is included for architecture completeness only.

```

{
  "image_id": "img_000887",
  "dpla_output": {
    "best_proof": {
      "proof_id": "p1",
      "score": 0.84,
      "steps": [
        "trait(img_000887, one_pair, 0.89)",
        "candidate_order(img_000887, diptera)",
        "trait(img_000887, piercing_sucking, 0.78)",
        "trait(img_000887, slender, 0.87)",
        "candidate_family(img_000887, culicidae)",
        "cnn_class(img_000887, mosquito, 0.74)",
        "final_class(img_000887, mosquito)"
      ]
    },
    "alternative_proofs": [
      {
        "proof_id": "p2",
        "final_class": "green_lacewing",
        "score": 0.17
      },
      {
        "proof_id": "p3",
        "final_class": "dragonfly",
        "score": 0.11
      }
    ]
  },
  "search_stats": {
    "approximate_inference": true,
    "proofs_considered": 9,
    "proofs_kept": 3
  }
}

```

```
}  
}  
}
```

Important note

This output does **not** mean DPLA* is the classifier.

It means DPLA* helps expose the strongest proofs when approximate inference is enabled. That matches its role as an inference method for neural probabilistic logic programming, not as a separate classification model.

15. End-to-End Example Summary

Input

A lateral image of a mosquito specimen.

CNN package

The model predicts:

- mosquito as top-1
- Diptera as strongest order
- one visible wing pair
- membranous wings
- piercing-sucking mouthparts
- slender body

DeepProbLog reasoning

The system uses those outputs as evidence and evaluates rules that support:

- Diptera
- then Culicidae-like candidate behavior
- then mosquito

Final output

The system returns:

- final class = mosquito
 - reasoning trace
 - activated rules
 - optional best-proof summary later
-

16. Team Responsibilities Under This Contract

Dataset person

Must preserve:

- `image_id`
- file path
- split
- target label
- basic metadata if available

CNN person

Must output:

- top-1
- top-k
- trait heads
- visibility flags
- quality block

DeepProbLog person

Must consume:

- structured evidence only
- not raw images

Must return:

- final prediction
- rule trace
- summary

Integration person

Must define:

- conversion from CNN JSON to reasoning facts
- validation of field names
- test cases using mock data

DPLA* person

For now:

- can consume the DeepProbLog reasoning structure
- does not block the MVP
- should align future proof-level outputs with this schema

17. Acceptance Criteria for Interface Contract v1

This contract is considered accepted if:

- all team members use the same `image_id`
- the CNN output matches the JSON schema
- DeepProbLog can run with mock evidence even before the real CNN is finished
- the mosquito case can be simulated end-to-end
- the final reasoning output contains both a class and a trace
- no module depends on undocumented fields

18. Practical Notes for Today's Meeting

Lo que yo les diría hoy a los chicos es esto:

1. **El CNN no va a entregar solo una clase**

Va a entregar una clase **más evidencia visual estructurada**.

2. **DeepProbLog no necesita esperar el modelo final**

Ya puede trabajar con mocks como el del mosquito.

3. **La integración fuerte del proyecto está aquí**

En convertir la salida probabilística del CNN en evidencia que sí pueda razonar el sistema lógico. Eso está exactamente alineado con la idea de DeepProbLog como lenguaje que incorpora deep learning mediante neural predicates.

4. **DPLA*** queda en arquitectura, pero no bloquea el MVP**

Su papel crece cuando el número de reglas, pruebas o clases aumente.

19. Recommended Immediate Next Files

Para que esto sí aterrice hoy mismo, yo crearía estos archivos:

- `mock_input_mosquito.json`
 - `mock_cnn_output_mosquito.json`
 - `mock_reasoner_input_mosquito.json`
 - `mock_reasoning_output_mosquito.json`
 - `mosquito_rules.pl`
 - `interface_contract_v1.md`
-

20. Final Note

This contract is intentionally conservative.

Its purpose is not to lock the final research design, but to create a **shared implementation target** for the first integration milestone.